
Algorithm 1 ToG

Require: Input x , LLM π , depth limit D_{max} sample limit N .

Initialize $E^0 \leftarrow$ Extract entities on x , $P \leftarrow []$, $M \leftarrow 0$.

while $D \leq D_{max}$ **do**

$R_{cand}^D, P_{cand} \leftarrow \text{Search}(x, E^{D-1}, P)$

$R^D, P \leftarrow \text{Prune}(\pi, x, R_{cand}^D, P_{cand})$

$E_{cand}^D, P_{cand} \leftarrow \text{Search}(x, E^{D-1}, R^D, P)$

$E^D, P \leftarrow \text{Prune}(\pi, x, E_{cand}^D, P_{cand})$

if Reasoning(π, x, P) **then**

 Generate(π, x, P)

break

end if

 Increment D by 1.

end while

Algorithm 2 ToG-R

Require: Input x , LLM π , depth limit D_{max} sample limit N .

Initialize $E^0 \leftarrow$ Extract entities on x , $P \leftarrow []$, $M \leftarrow 0$.

while $D \leq D_{max}$ **do**

$R_{cand}^D, P_{cand} \leftarrow \text{Search}(x, E^{D-1}, P)$

$R^D, P \leftarrow \text{Prune}(\pi, x, R_{cand}^D, P_{cand})$

$E_{cand}^D, P_{cand} \leftarrow \text{Search}(x, E^{D-1}, R^D, P)$

if Reasoning(π, x, P, E_{cand}^D) **then**

 Generate(π, x, P, E_{cand}^D)

break

end if

$E^D, P \leftarrow \text{Random_Prune}(E_{cand}^D, P_{cand})$

 Increment D by 1.

end while